

Aula 10 - Design Patterns

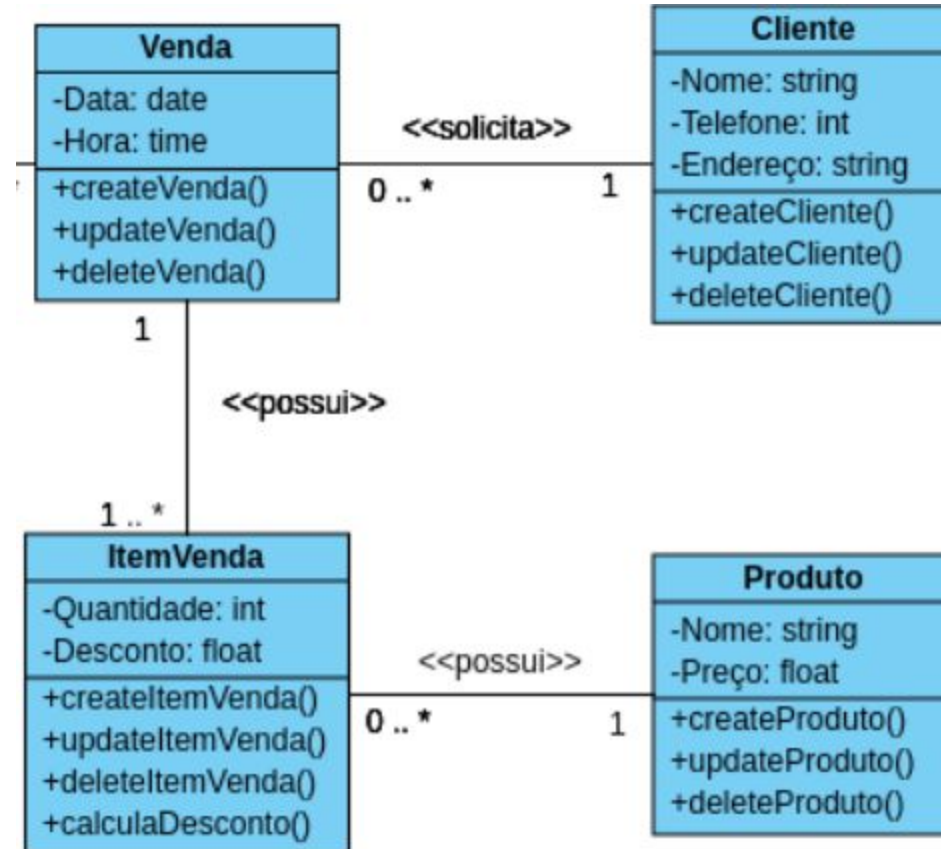
Disciplina: Arquitetura de Software

Prof. Me. João Paulo Biazotto

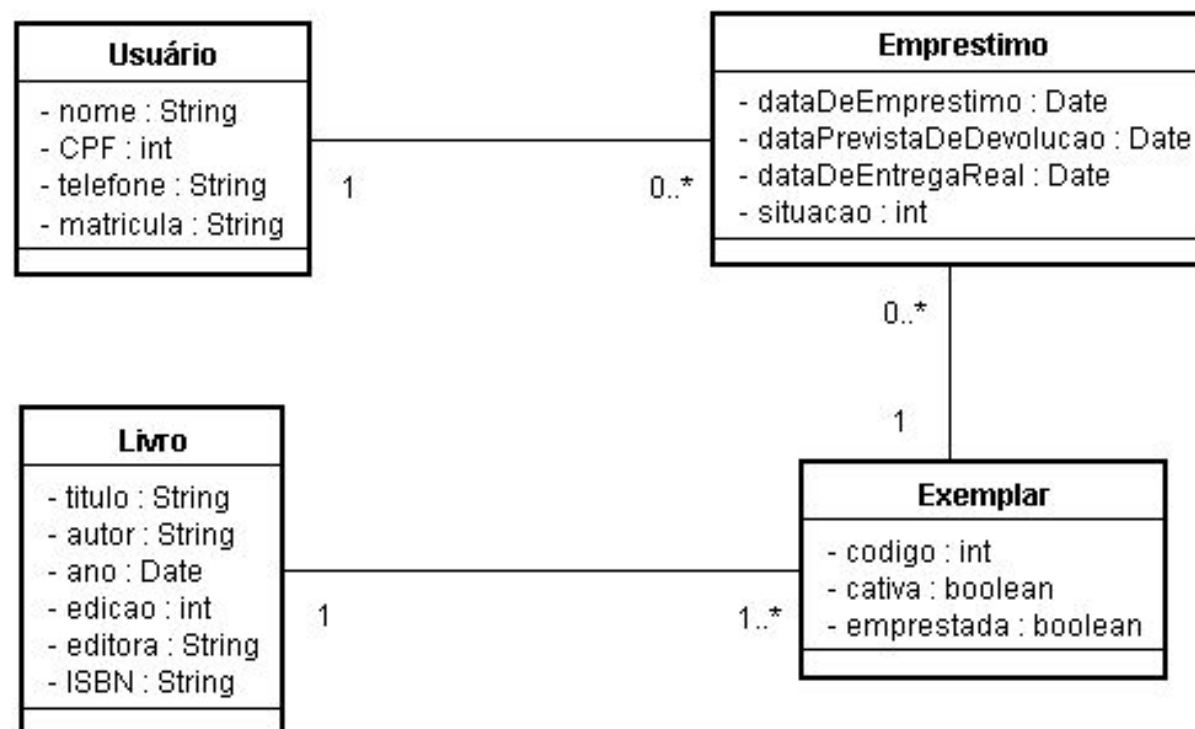
Problemática e Motivação

- Vários problemas de design de software se repetem em diferentes projetos;
- Milhares (ou até milhões) de programadores planejam e executam soluções para esses problemas;

Sistema de Vendas



Sistema de Bibliotecas



Problemática e Motivação

- Tais semelhanças também ocorrem na implementação dos sistemas;
- Uma venda pode ter muitas formas de pagamento, cada uma com uma especificidade:
 - Boleto;
 - Cartão de Crédito;
 - Cartão de Débito;

Problemática e Motivação

- Um sistema de notificação pode enviar mensagens por diferentes canais:
 - Instagram;
 - Whatsapp;
 - SMS;
- O mesmo sistema usando diferentes ***estratégias***.
- Qual seria a forma mais eficiente de implementar isso?

Problemática e Motivação

- Precisamos sempre implementar uma nova solução para os mesmos problemas?



Design Patterns

- Em 1994, quatro autores (que ficaram conhecidos como *Gang of Four* - *GoF*) perceberam que alguns desafios do design de software eram comuns;
- Vários desenvolvedores já haviam proposto soluções para tais problemas, e tais soluções poderiam ser re-usadas.
- Eles então catalogaram tais soluções e escreveram um livro;

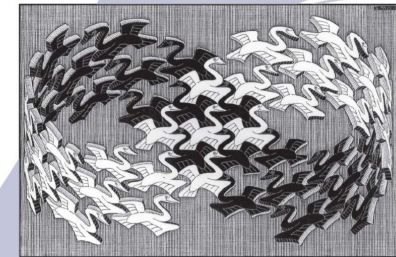
Design Patterns

- O catálogo de padrões GoF é uma referência até hoje para o desenvolvimento eficiente de sistemas orientados a objetos;
- Eles descrevem 23 padrões de projetos.

Design Patterns

Elements of Reusable
Object-Oriented Software

Erich Gamma
Richard Helm
Ralph Johnson
John Vlissides



Cover art © 1994 M.C. Escher / Cordon Art - Baarn - Holland. All rights reserved.

Foreword by Grady Booch



O que são Design Patterns?

Design Patterns (ou padrões de projeto) são soluções reutilizáveis para problemas recorrentes no design de software. Eles fornecem um modelo comprovado para estruturar código de forma eficiente e escalável.

Por que usar Design Patterns?

- Melhor organização do código
- Maior reutilização
- Facilidade de manutenção
- Padrões amplamente conhecidos e documentados

Benefícios

- Reduzem a complexidade do código
- Facilitam a comunicação entre desenvolvedores
- Melhoram a testabilidade e extensibilidade

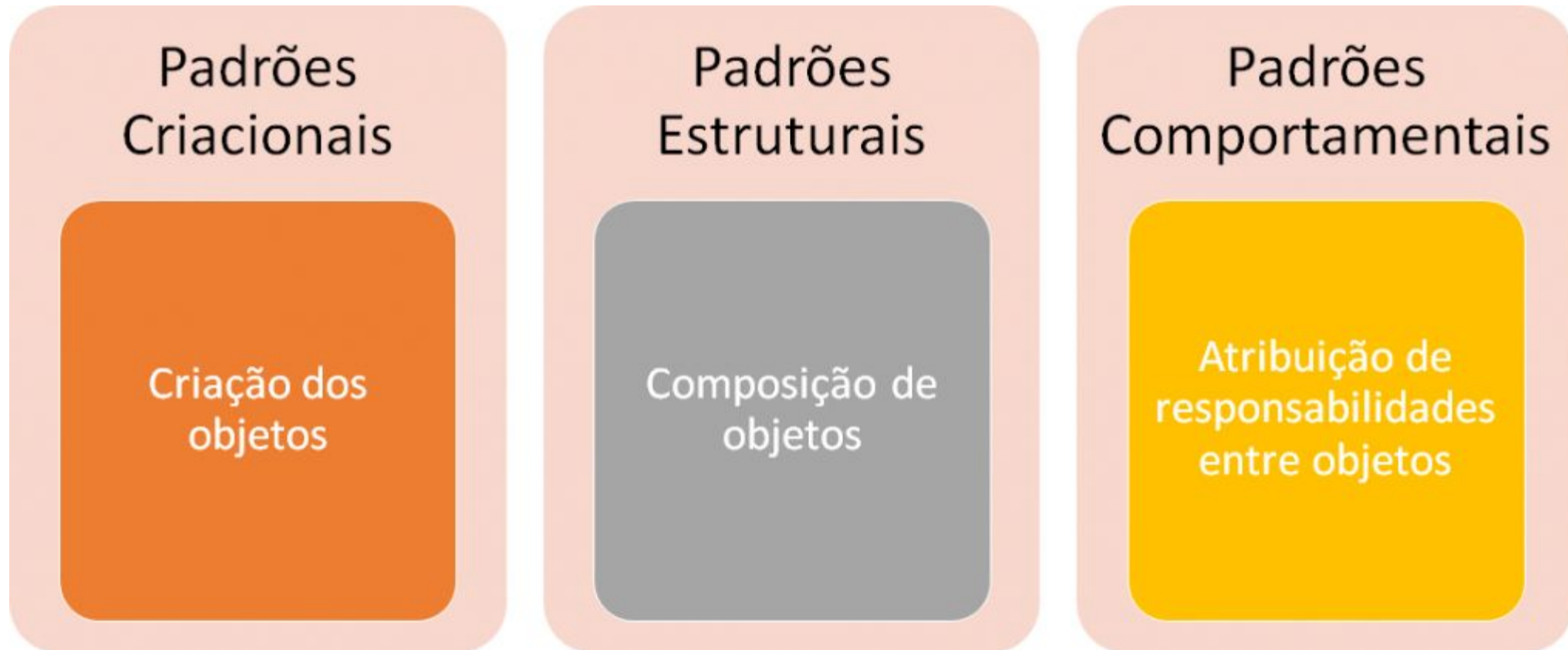
Limitações

- Requerem conhecimento prévio para aplicação correta
- Podem ser desnecessários para problemas simples
- Má implementação pode gerar código mais complexo

Como os Design Patterns se encaixam na Arquitetura de Software?

- Auxiliam na criação de sistemas modulares
- Reduzem o acoplamento entre componentes
- Melhoram a manutenção dos sistemas.

Categorias de Design Patterns



Categorias de Design Patterns

- Padrões Criacionais: *Focados na criação de objetos de forma eficiente e flexível.*
- Padrões Estruturais: *Auxiliam na organização de classes e objetos para formar estruturas maiores.*
- Padrões Comportamentais: *Definem como os objetos interagem e comunicam entre si.*

Padrões comportamentais

Padrões **comportamentais** se concentram em como os objetos **interagem** e como **responsabilidades** são distribuídas entre eles.

Padrões comportamentais

- Promover uma comunicação **flexível** e eficiente entre objetos.
- Evitar dependências **rígidas** e lógica de controle espalhada.

Padrões comportamentais

- Favorecem o baixo **acoplamento** e a alta coesão.
- Facilitam a manutenção, extensão e reutilização de código.

Padrões Comportamentais

- ***Template Method:*** Define o esqueleto de um algoritmo, deixando passos específicos para as subclasses.
- ***Mediator*** : Centraliza a comunicação entre objetos para reduzir dependências diretas.
- ***State:*** Permite que um objeto altere seu comportamento quando seu estado muda.

Padrões Comportamentais

- ***Observer***: Notifica múltiplos objetos quando o estado de um objeto muda.
- ***Command***: Encapsula uma solicitação como um objeto.
- ***Iterator***: Fornece uma forma de acessar os elementos de uma coleção sequencialmente.
- ***Strategy***: Permite alterar o algoritmo de uma tarefa em tempo de execução.

Padrões Comportamentais

- ***Observer***: Notifica múltiplos objetos quando o estado de um objeto muda.
- ***Command***: Encapsula uma solicitação como um objeto.
- ***Iterator***: Fornece uma forma de acessar os elementos de uma coleção sequencialmente.
- ***Strategy***: Permite alterar o algoritmo de uma tarefa em tempo de execução.

Problemática

- Você tem um sistema de vendas que oferece, inicialmente, apenas um tipo de pagamento.
- Assim, você implementa a lógica de pagamento na classe de venda.
- No entanto, a medida que o software cresce, você quer prover mais métodos de pagamento.

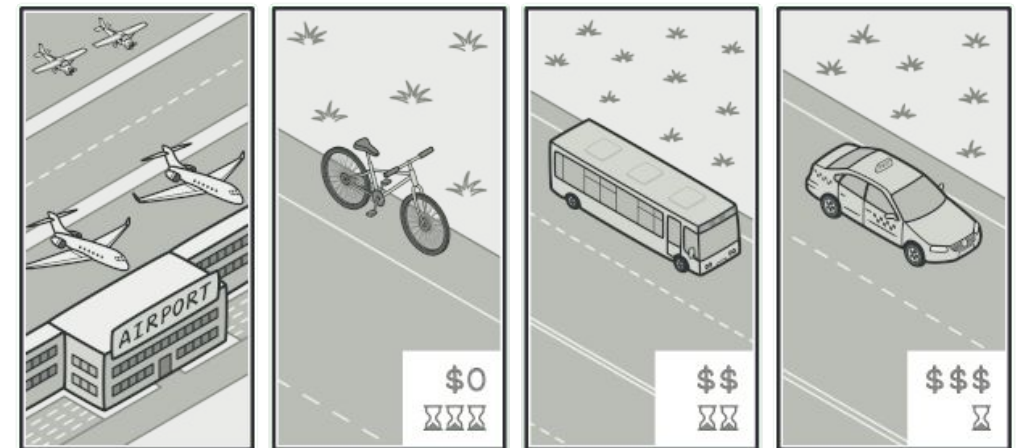
Problemática

- Qualquer **alteração** na lógica de pagamento — seja uma simples correção de bug ou um pequeno ajuste na pontuação de ruas — **afeta toda a classe.**
- O que fazer?



Padrão Strategy

- O Strategy Pattern propõe **mover algoritmos que variam para classes separadas**, chamadas de estratégias, permitindo que sejam intercambiáveis em tempo de execução.



Padrão Strategy

- Context: Classe principal que utiliza uma estratégia.
- Strategy (Interface): Define um contrato comum para todas as estratégias.
- Concrete Strategies: Implementações específicas do algoritmo.

Padrão Strategy

- Cálculos de desconto em carrinho de compras.
- Algoritmos de ordenação configuráveis.
- Estratégias de autenticação (senha, biometria, OAuth...).

Bora programar: Implementação do Padrão Strategy



DÚVIDAS?

Aula 09 - Design Patterns

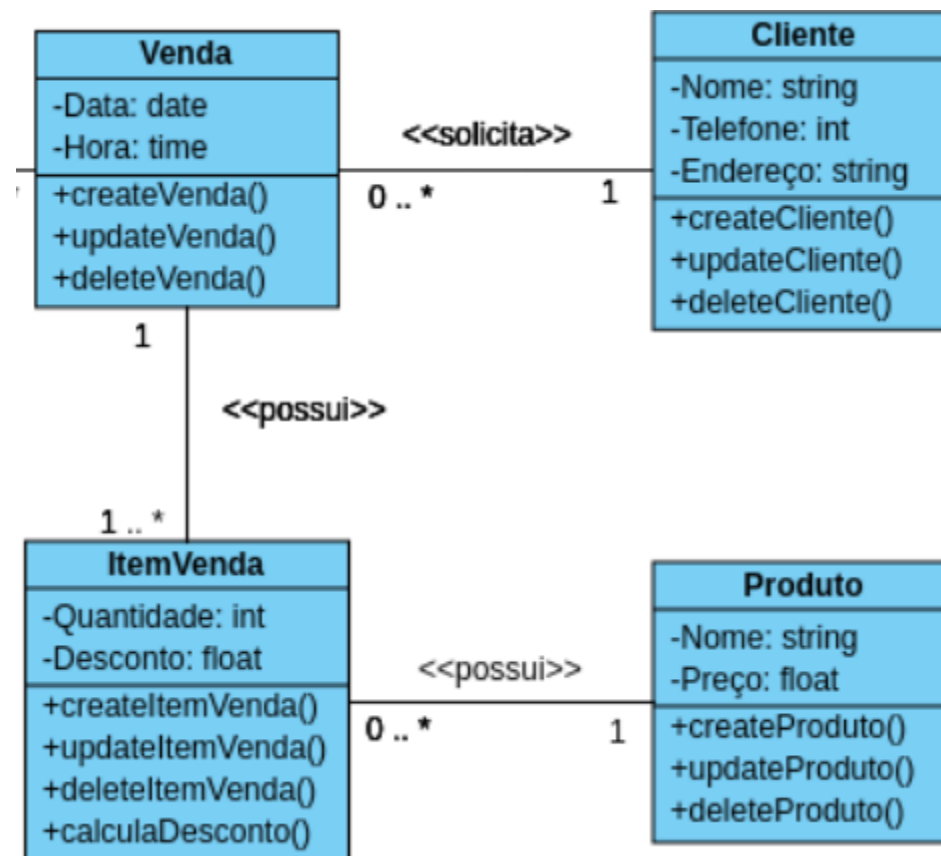
Disciplina: Arquitetura de Software

Prof. Me. João Paulo Biazotto

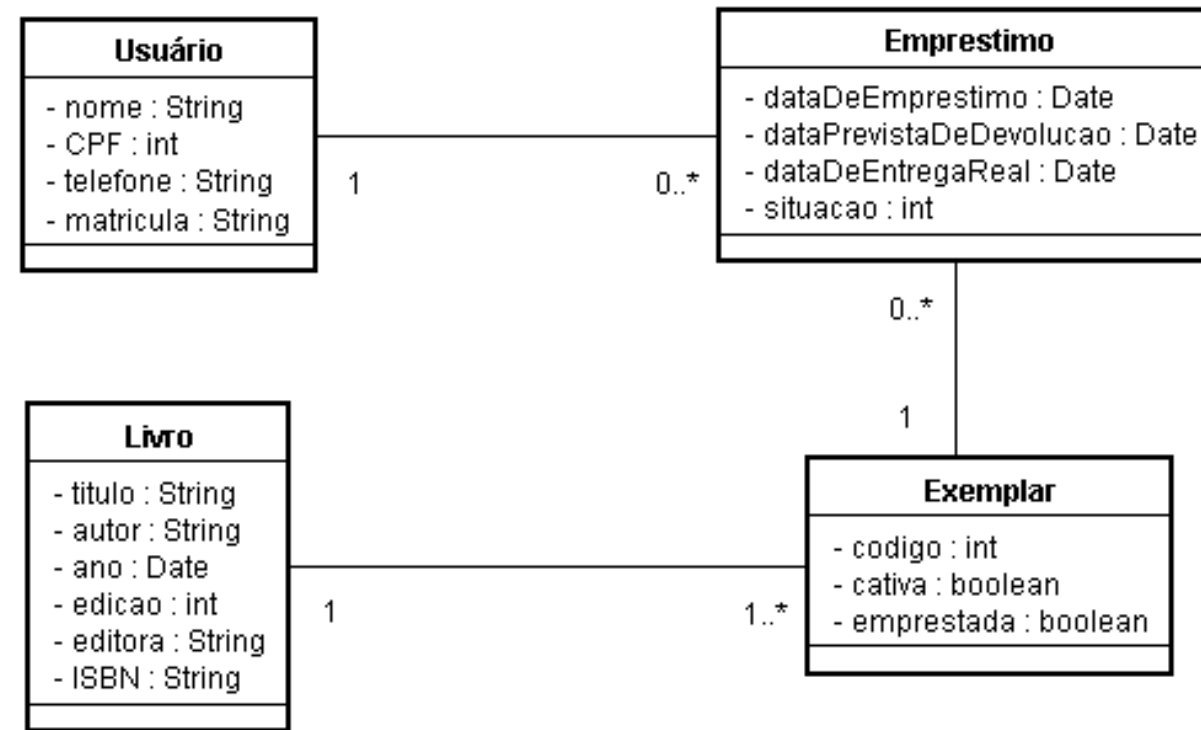
Problemática e Motivação

- Vários problemas de design de software se repetem em diferentes projetos;
- Milhares (ou até milhões) de programadores planejam e executam soluções para esses problemas;

Sistema de Vendas



Sistema de Bibliotecas



Problemática e Motivação

- Tais semelhanças também ocorrem na implementação dos sistemas;
- Uma venda pode ter muitas formas de pagamento, cada uma com uma especificidade:
 - Boleto;
 - Cartão de Crédito;
 - Cartão de Débito;

Problemática e Motivação

- Um sistema de notificação pode enviar mensagens por diferentes canais:
 - Instagram;
 - Whatsapp;
 - SMS;
- O mesmo sistema usando diferentes *estratégias*.
- Qual seria a forma mais eficiente de implementar isso?

Problemática e Motivação

- Precisamos sempre implementar uma nova solução para os mesmos problemas?



Design Patterns

- Em 1994, quatro autores (que ficaram conhecidos como *Gang of Four* - *GoF*) perceberam que alguns desafios do design de software eram comuns;
- Vários desenvolvedores já haviam proposto soluções para tais problemas, e tais soluções poderiam ser re-usadas.
- Eles então catalogaram tais soluções e escreveram um livro;

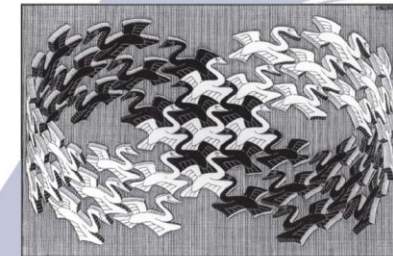
Design Patterns

- O catálogo de padrões GoF é uma referência até hoje para o desenvolvimento eficiente de sistemas orientados a objetos;
- Eles descrevem 23 padrões de projetos.

Design Patterns

Elements of Reusable
Object-Oriented Software

Erich Gamma
Richard Helm
Ralph Johnson
John Vlissides



Cover art © 1994 M.C. Escher / Cordon Art - Baarn - Holland. All rights reserved.

Foreword by Grady Booch



O que são Design Patterns?

Design Patterns (ou padrões de projeto) são soluções reutilizáveis para problemas recorrentes no design de software. Eles fornecem um modelo comprovado para estruturar código de forma eficiente e escalável.

Por que usar Design Patterns?

- Melhor organização do código
- Maior reutilização
- Facilidade de manutenção
- Padrões amplamente conhecidos e documentados

Benefícios

- Reduzem a complexidade do código
- Facilitam a comunicação entre desenvolvedores
- Melhoram a testabilidade e extensibilidade

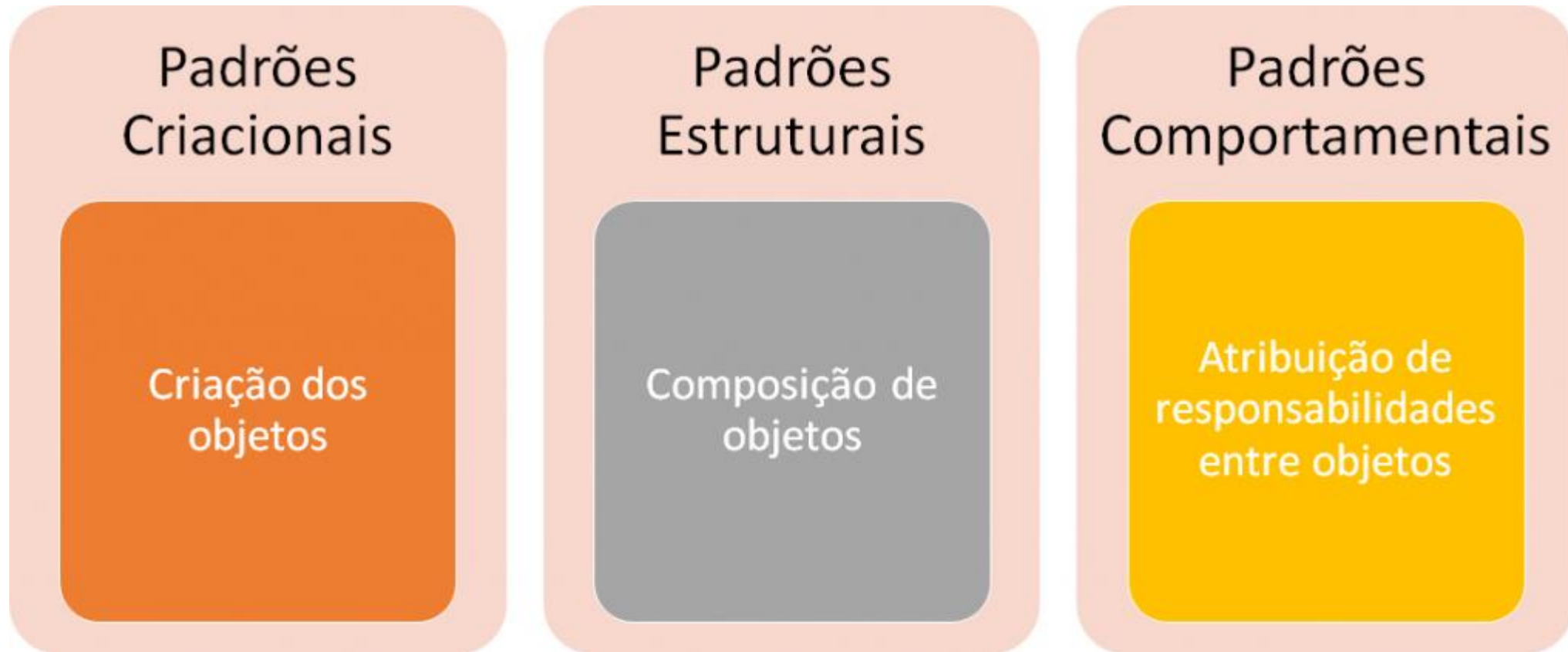
Limitações

- Requerem conhecimento prévio para aplicação correta
- Podem ser desnecessários para problemas simples
- Má implementação pode gerar código mais complexo

Como os Design Patterns se encaixam na Arquitetura de Software?

- Auxiliam na criação de sistemas modulares
- Reduzem o acoplamento entre componentes
- Melhoram a manutenção dos sistemas.

Categorias de Design Patterns



Categorias de Design Patterns

- Padrões Criacionais: *Focados na criação de objetos de forma eficiente e flexível.*
- Padrões Estruturais: *Auxiliam na organização de classes e objetos para formar estruturas maiores.*
- Padrões Comportamentais: *Definem como os objetos interagem e comunicam entre si.*

Padrões criacionais

Os padrões de projeto criacionais **abstraem o processo de instanciação**. Eles ajudam a tornar um sistema independente de como seus objetos são criados, compostos e representados.

Padrões criacionais

Existem dois temas recorrentes nesses padrões. Primeiro, todos eles **encapsulam o conhecimento** sobre quais classes concretas o sistema utiliza. Segundo, eles **ocultam como as instâncias** dessas classes são **criadas e combinadas**.

Padrões criacionais

Consequentemente, os padrões criacionais oferecem grande **flexibilidade** em relação ao *que* é criado, *quem* o cria, *como* é criado e *quando*. Eles permitem configurar um sistema com objetos que podem variar amplamente em *estrutura* e *funcionalidade*.

Padrões criacionais

- **Singleton:** *garante que uma classe tenha apenas uma instância;*
- **Factory Method:** *define uma interface para criar objetos;*
- **Abstract Factory:** *define uma interface para criar famílias de objetos;*
- **Builder:** *permite a construção de um objeto complexo passo a passo;*
- **Prototype:** *cria objetos a partir de um protótipo.*

Padrões criacionais

- **Singleton:** *garante que uma classe tenha apenas uma instância;*
- **Factory Method:** *define uma interface para criar objetos;*
- **Abstract Factory:** *define uma interface para criar famílias de objetos;*
- **Builder:** *permite a construção de um objeto complexo passo a passo;*
- **Prototype:** *cria objetos a partir de um protótipo.*

Problemática

- Um sistema possui um banco de dados, no qual vários métodos podem escrever;
- Problemas:
 - Vários métodos abrem uma conexão com banco de dados -> Problemas de performance
 - Vários métodos tentam ler/escrever dados ao mesmo tempo no banco -> Problemas de integridade;

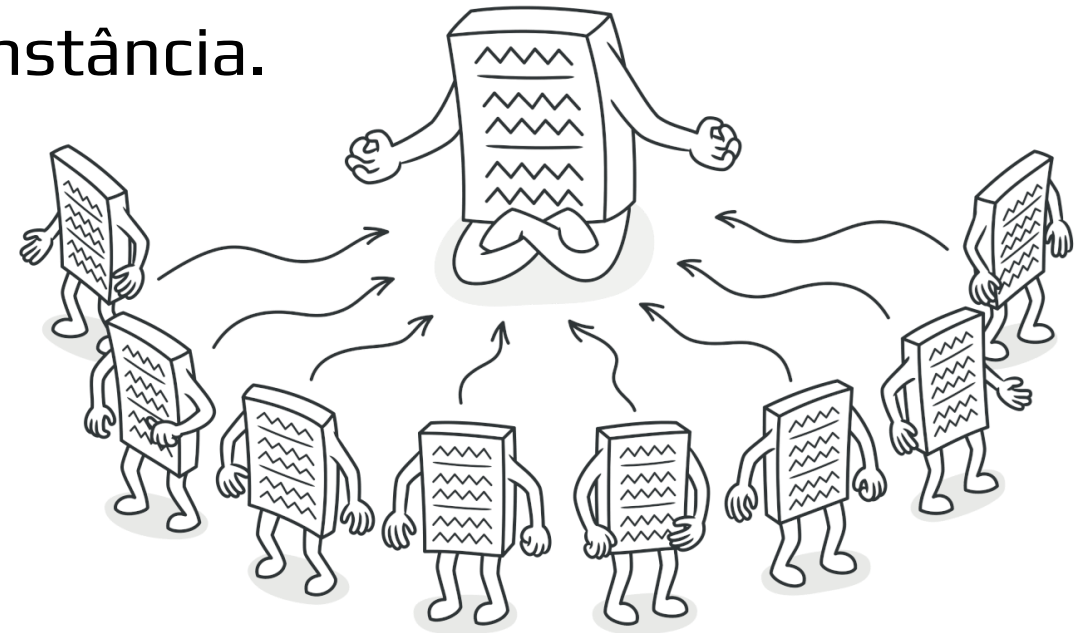
Problemática

- O problema aqui é que vários métodos estão compartilhando o mesmo recurso, e precisamos limitar isso;
- O que fazer?



Padrão Singleton

O **Singleton** é um padrão de projeto criacional que permite a você **garantir que uma classe tenha apenas uma instância**, enquanto provê um ponto de acesso global para essa instância.



Padrão Singleton

Útil em situações onde você deseja **compartilhar o mesmo objeto** em todo o sistema, como gerenciadores de **log**, **conexões de banco de dados**, caches, configurações globais, entre outros.

Bora programar: Implementação do Padrão Singleton



Atividade

Baseado na implementação vista em sala, e tendo como base o projeto que vem sendo desenvolvido para a Escola de T.I, implemente um logger no seu projeto. Esse logger deve registrar em arquivo todas as ações que um usuário fizer com o sistema.

Padrões criacionais

- ~~**Singleton:** *garante que uma classe tenha apenas uma instância;*~~
- **Factory Method:** *define uma interface para criar objetos;*
- **Abstract Factory:** *define uma interface para criar famílias de objetos;*
- **Builder:** *permite a construção de um objeto complexo passo a passo;*
- **Prototype:** *cria objetos a partir de um protótipo.*

Problemática

- Você está criando um leitor de documentos;
- A princípio, o leitor trabalha apenas com documentos do tipo txt;
- Ao longo do tempo, novos tipos de documentos precisam ser suportados, porém não sabemos exatamente quais tipos;

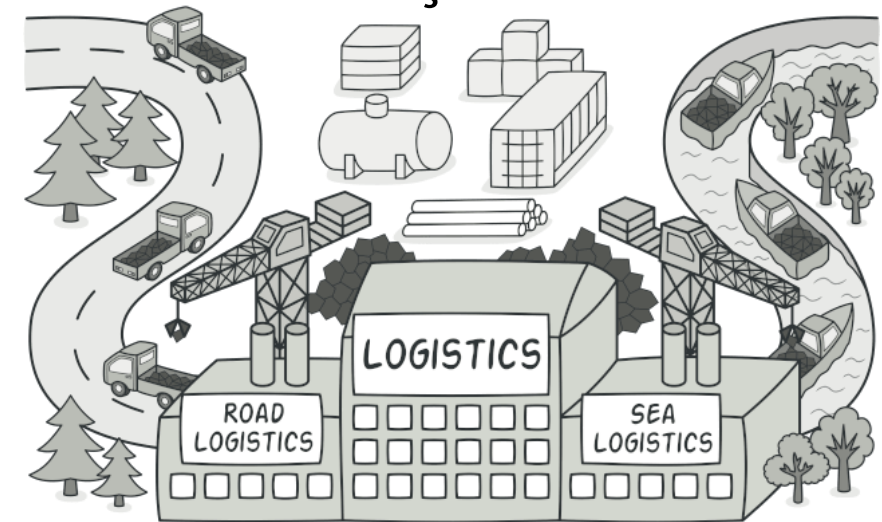
Problemática

- O problema aqui é a cada novo tipo de documento que precisa ser suportado, várias partes do código precisam ser alterados:
- O que fazer?



Padrão Factory Method

O **Factory Method** define uma interface para a criação de um objeto, mas permite que as **subclasses decidam qual classe instanciar**. Esse padrão permite que uma classe delegue a instanciação às suas subclasses.



Padrão Factory Method

- Uma classe não pode antecipar a **classe dos objetos** que deve criar.
- Uma classe deseja que suas **subclasses especifiquem os objetos** que ela cria.
- As classes delegam a responsabilidade para uma entre várias **subclasses auxiliares**, e você quer localizar o conhecimento sobre qual subclasse auxiliar é a responsável.

Bora programar: Implementação do Padrão Factory Method



Atividade

Baseado na implementação vista em sala, e tendo como base o projeto que vem sendo desenvolvido para a Escola de T.I, utilize o design pattern Factory Method no seu projeto. Tente escolher uma aplicação simples, direta e pequena para entender como o padrão pode ser usado.

DÚVIDAS?



Manutenção de Software

Código Limpo – Funções



- O que faz o trecho de código a seguir?

Código Limpo – Funções



```
public static String testableHtml(
    PageData pageData,
    boolean includeSuiteSetup
) throws Exception {
    WikiPage wikiPage = pageData.getWikiPage();
    StringBuffer buffer = new StringBuffer();
    if (pageData.hasAttribute("Test")) {
        if (includeSuiteSetup) {
            WikiPage suiteSetup =
                PageCrawlerImpl.getInheritedPage(
                    SuiteResponder.SUITE_SETUP_NAME, wikiPage
                );
            if (suiteSetup != null) {
                WikiPagePath pagePath =
                    suiteSetup.getPageCrawler().getFullPath(suiteSetup);
                String pagePathName = PathParser.render(pagePath);
                buffer.append("!include -setup .")
                    .append(pagePathName)
                    .append("\n");
            }
        }
        WikiPage setup =
            PageCrawlerImpl.getInheritedPage("SetUp", wikiPage);
        if (setup != null) {
            WikiPagePath setupPath =
                wikiPage.getPageCrawler().getFullPath(setup);
            String setupPathName = PathParser.render(setupPath);
            buffer.append("!include -setup .")
                .append(setupPathName)
                .append("\n");
        }
    }
}
```

```
buffer.append(pageData.getContent());
if (pageData.hasAttribute("Test")) {
    WikiPage teardown =
        PageCrawlerImpl.getInheritedPage("TearDown", wikiPage);
    if (teardown != null) {
        WikiPagePath teardownPath =
            wikiPage.getPageCrawler().getFullPath(teardown);
        String teardownPathName = PathParser.render(teardownPath);
        buffer.append("\n")
            .append("!include -teardown .")
            .append(teardownPathName)
            .append("\n");
    }
    if (includeSuiteSetup) {
        WikiPage suiteTeardown =
            PageCrawlerImpl.getInheritedPage(
                SuiteResponder.SUITE_TEARDOWN_NAME,
                wikiPage
            );
        if (suiteTeardown != null) {
            WikiPagePath pagePath =
                suiteTeardown.getPageCrawler().getFullPath(suiteTeardown);
            String pagePathName = PathParser.render(pagePath);
            buffer.append("!include -teardown .")
                .append(pagePathName)
                .append("\n");
        }
    }
}
pageData.setContent(buffer.toString());
return pageData.getHtml();
}
```

Funções



- Muito conteúdo (append, crawler, parser, buffer, etc)
- Código duplicado
- Condicionais aninhadas

Funções



- Refatorando
- E agora?

```
public static String renderPageWithSetupsAndTeardowns(  
    PageData pageData, boolean isSuite  
) throws Exception {  
    boolean isTestPage = pageData.hasAttribute("Test");  
    if (isTestPage) {  
        WikiPage testPage = pageData.getWikiPage();  
        StringBuffer newPageContent = new StringBuffer();  
        includeSetupPages(testPage, newPageContent, isSuite);  
        newPageContent.append(pageData.getContent());  
        includeTeardownPages(testPage, newPageContent, isSuite);  
        pageData.setContent(newPageContent.toString());  
    }  
  
    return pageData.getHtml();  
}
```

Funções



- Setups e teardown em página de testes
- Renderiza HTML
- Teste web
- Extrair informação da segunda função é mais fácil que da primeira

Funções



- O que torna uma função fácil de ler?

Funções



- Como podemos fazer uma função comunicar sua intenção?

Funções



- Quais atributos uma função deve ter?

Funções devem ser pequenas



- Primeira regra
- Idealmente: if, else, whiles devem ter uma linha

```
public static String renderPageWithSetupsAndTeardowns(  
    PageData pageData, boolean isSuite  
) throws Exception {  
    boolean isTestPage = pageData.hasAttribute("Test");  
    if (isTestPage) {  
        WikiPage testPage = pageData.getWikiPage();  
        StringBuffer newPageContent = new StringBuffer();  
        includeSetupPages(testPage, newPageContent, isSuite);  
        newPageContent.append(pageData.getContent());  
        includeTeardownPages(testPage, newPageContent, isSuite);  
        pageData.setContent(newPageContent.toString());  
    }  
  
    return pageData.getHtml();  
}
```

Funções devem ser pequenas



```
public static String renderPageWithSetupsAndTeardowns(  
    PageData pageData, boolean isSuite  
) throws Exception {  
    boolean isTestPage = pageData.hasAttribute("Test");  
    if (isTestPage) {  
        WikiPage testPage = pageData.getWikiPage();  
        StringBuffer newPageContent = new StringBuffer();  
        includeSetupPages(testPage, newPageContent, isSuite);  
        newPageContent.append(pageData.getContent());  
        includeTeardownPages(testPage, newPageContent, isSuite);  
        pageData.setContent(newPageContent.toString());  
    }  
  
    return pageData.getHtml();  
}
```

```
public static String renderPageWithSetupsAndTeardowns(  
    PageData pageData, boolean isSuite  
) throws Exception {  
    boolean isTestPage = pageData.hasAttribute("Test");  
    if (isTestPage(pageData))  
        includeSetupAndTeardownPages(pageData, isSuite);  
    return pageData.getHtml();  
}
```

Funções devem realizar apenas uma tarefa



- A primeira função realiza várias tarefas
- Cria buffers
- Busca páginas
- Busca heranças
- Faz parser
- Acrescenta string
- Gera HTML

Funções devem realizar apenas uma tarefa



- Possui vários níveis de abstração
- Alto nível: `getHtml()`
- Nível intermediário: `PathParser.render(PagePath)`
- Baixo nível: `append("\n")`

Funções devem realizar apenas uma tarefa



- A última função faz apenas uma coisa
- Adiciona setups e teardowns em páginas de testes

```

public static String testableHtml(
    PageData pageData,
    boolean includeSuiteSetup
) throws Exception {
    WikiPage wikiPage = pageData.getWikiPage();
    StringBuffer buffer = new StringBuffer();
    if (pageData.hasAttribute("Test")) {
        if (includeSuiteSetup) {
            WikiPage suiteSetup =
                PageCrawlerImpl.getInheritedPage(
                    SuiteResponder.SUITE_SETUP_NAME, wikiPage
                );
            if (suiteSetup != null) {
                WikiPagePath pagePath =
                    suiteSetup.getPageCrawler().getFullPath(suiteSetup);
                String pagePathName = PathParser.render(pagePath);
                buffer.append("!include -setup .")
                    .append(pagePathName)
                    .append("\n");
            }
        }
        WikiPage setup =
            PageCrawlerImpl.getInheritedPage("SetUp", wikiPage);
        if (setup != null) {
            WikiPagePath setupPath =
                wikiPage.getPageCrawler().getFullPath(setup);
            String setupPathName = PathParser.render(setupPath);
            buffer.append("!include -setup .")
                .append(setupPathName)
                .append("\n");
        }
    }
}

```

```

buffer.append(pageData.getContent());
if (pageData.hasAttribute("Test")) {
    WikiPage teardown =
        PageCrawlerImpl.getInheritedPage("TearDown", wikiPage);
    if (teardown != null) {
        WikiPagePath teardownPath =
            wikiPage.getPageCrawler().getFullPath(teardown);
        String teardownPathName = PathParser.render(teardownPath);
        buffer.append("\n")
            .append("!include -teardown .")
            .append(teardownPathName)
            .append("\n");
    }
    if (includeSuiteSetup) {
        WikiPage suiteTeardown =
            PageCrawlerImpl.getInheritedPage(
                SuiteResponder.SUITE_TEARDOWN_NAME,
                wikiPage
            );
        if (suiteTeardown != null) {
            WikiPagePath pagePath =
                suiteTeardown.getPageCrawler().getFullPath(suiteTeardown);
            String pagePathName = PathParser.render(pagePath);
            buffer.append("!include -teardown .")
                .append(pagePathName)
                .append("\n");
        }
    }
}
pageData.setContent(buffer.toString());
return pageData.getHtml();
}

```

```

public static String renderPageWithSetupsAndTeardowns(
    PageData pageData, boolean isSuite
) throws Exception {
    boolean isTestPage = pageData.hasAttribute("Test");
    if (isTestPage(pageData))
        includeSetupAndTeardownPages(pageData, isSuite);
    return pageData.getHtml();
}

```

Nomes em funções



- Nomes descritivos são melhores que nomes curtos e enigmáticos
- Definir um nome descritivo faz com que um comentário seja desnecessário

Nomes em funções



- Nomear entidades não é uma tarefa fácil (apesar de parecer fácil)
 - IDEs facilitam a troca de nomes
- Seja consistente nos nomes
 - Utilizar mesmo prefixo/sufixo

Argumentos em funções



- O ideal é não utilizar
- Se necessário: um ou dois
- Evitar três ou mais

Argumentos em funções



- Por que evitar?
- Torna a função mais complexa
- Dificulta a criação de testes (todas as combinações devem ser testadas)

Um argumento



- Realizar uma pergunta sobre o argumento
- `boolean fileExists("MyFile.txt")`

Um argumento



- Operar no argumento, transformar e retornar ele
- `InputStream fileOpen("MyFile.txt")`

Um argumento



- Alterar o estado do sistema
- void passwordAttemptFailedNtimes(int attempts)

Um argumento



- void transform(StringBuffer out)
- ou
- StringBuffer transform(StringBuffer in)

Um argumento



- void transform(StringBuffer out)
- Pode confundir entrada com saída
- StringBuffer transform(StringBuffer in)
- Retorna o valor correto, dado a entrada

Efeitos colaterais



- Evitar funções que fazem tarefas “escondidas”
- Exemplos:
 - Mudanças inesperadas em variáveis, parâmetros ou atributos
 - Realização de tarefas distintas do seu objetivo

Efeitos colaterais



- Qual o efeito colateral da seguinte função?

```
public class UserValidator {  
    private Cryptographer cryptographer;  
  
    public boolean checkPassword(String userName, String password) {  
        User user = UserGateway.findByName(userName);  
        if (user != User.NULL) {  
            String codedPhrase = user.  
                getPhraseEncodedByPassword();  
            String phrase = cryptographer.decrypt(codedPhrase, password);  
            if ("Valid Password".equals(phrase)) {  
                Session.initialize();  
                return true;  
            }  
        }  
        return false;  
    }  
}
```

Efeitos colaterais



- Qual o efeito colateral da seguinte função?

```
public class UserValidator {  
    private Cryptographer cryptographer;  
  
    public boolean checkPassword(String userName, String password) {  
        User user = UserGateway.findByName(userName);  
        if (user != User.NULL) {  
            String codedPhrase = user.  
                getPhraseEncodedByPassword();  
            String phrase = cryptographer.decrypt(codedPhrase, password);  
            if ("Valid Password".equals(phrase)) {  
                Session.initialize();  
                return true;  
            }  
        }  
        return false;  
    }  
}
```

A red arrow points from the left side of the slide to the line of code `Session.initialize();` within the `checkPassword` method, highlighting it as the source of the side effect.

Efeitos colaterais



- De acordo com o nome, a função deveria validar o password, no entanto também inicializa a sessão

```
public class UserValidator {  
    private Cryptographer cryptographer;  
  
    public boolean checkPassword(String userName, String password) {  
        User user = UserGateway.findByName(userName);  
        if (user != User.NULL) {  
            String codedPhrase = user.  
                getPhraseEncodedByPassword();  
            String phrase = cryptographer.decrypt(codedPhrase, password);  
            if ("Valid Password".equals(phrase)) {  
                Session.initialize();  
                return true;  
            }  
        }  
        return false;  
    }  
}
```

- Renomear: **checkPasswordAndInitializeSession()**

Funções e repetições



- Não repita a si mesmo
- Duplicação só traz problemas
- Vários princípios e práticas foram criados para evitar ou eliminar duplicação de código
- Funções, métodos, OO, programação estruturada, programação orientada a aspectos, em parte são, estratégias para eliminar duplicação

Onde está a duplicação?



```
public static String testableHtml(
    PageData pageData,
    boolean includeSuiteSetup
) throws Exception {
    WikiPage wikiPage = pageData.getWikiPage();
    StringBuffer buffer = new StringBuffer();
    if (pageData.hasAttribute("Test")) {
        if (includeSuiteSetup) {
            WikiPage suiteSetup =
                PageCrawlerImpl.getInheritedPage(
                    SuiteResponder.SUITE_SETUP_NAME, wikiPage
                );
            if (suiteSetup != null) {
                WikiPagePath pagePath =
                    suiteSetup.getPageCrawler().getFullPath(suiteSetup);
                String pagePathName = PathParser.render(pagePath);
                buffer.append("!include -setup .")
                    .append(pagePathName)
                    .append("\n");
            }
        }
        WikiPage setup =
            PageCrawlerImpl.getInheritedPage("SetUp", wikiPage);
        if (setup != null) {
            WikiPagePath setupPath =
                wikiPage.getPageCrawler().getFullPath(setup);
            String setupPathName = PathParser.render(setupPath);
            buffer.append("!include -setup .")
                .append(setupPathName)
                .append("\n");
        }
    }
}
```

```
buffer.append(pageData.getContent());
if (pageData.hasAttribute("Test")) {
    WikiPage teardown =
        PageCrawlerImpl.getInheritedPage("TearDown", wikiPage);
    if (teardown != null) {
        WikiPagePath teardownPath =
            wikiPage.getPageCrawler().getFullPath(teardown);
        String teardownPathName = PathParser.render(teardownPath);
        buffer.append("\n")
            .append("!include -teardown .")
            .append(teardownPathName)
            .append("\n");
    }
    if (includeSuiteSetup) {
        WikiPage suiteTeardown =
            PageCrawlerImpl.getInheritedPage(
                SuiteResponder.SUITE_TEARDOWN_NAME,
                wikiPage
            );
        if (suiteTeardown != null) {
            WikiPagePath pagePath =
                suiteTeardown.getPageCrawler().getFullPath(suiteTeardown);
            String pagePathName = PathParser.render(pagePath);
            buffer.append("!include -teardown .")
                .append(pagePathName)
                .append("\n");
        }
    }
}
pageData.setContent(buffer.toString());
return pageData.getHtml();
}
```

Onde está a duplicação?



- 4 locais para mudar – MANUTENÇÃO
- Não é fácil de ver
- 4 oportunidades de introduzir erros

Conclusões



- Funções devem ser refinadas ao longo do tempo
- Primeira versão nunca é perfeita
 - Geralmente, longa e complicada
 - Talvez com loops aninhados e longas listas de argumentos
 - Possivelmente com nomes ruins e código duplicado

Conclusões



- **Escrever testes**

- Faz com que as funções possam ser refinadas, divididas, renomeadas, removidas as duplicações, com a garantia dos testes



Manutenção de Software

Código Limpo – Formatação



- Formatação é importante
- Formatação de código trata de comunicação
- Comunicação é fundamental para desenvolvedores e profissionais

Formatação



- O que podemos fazer para melhorar a formatação de um código?

Dicas de Formatação

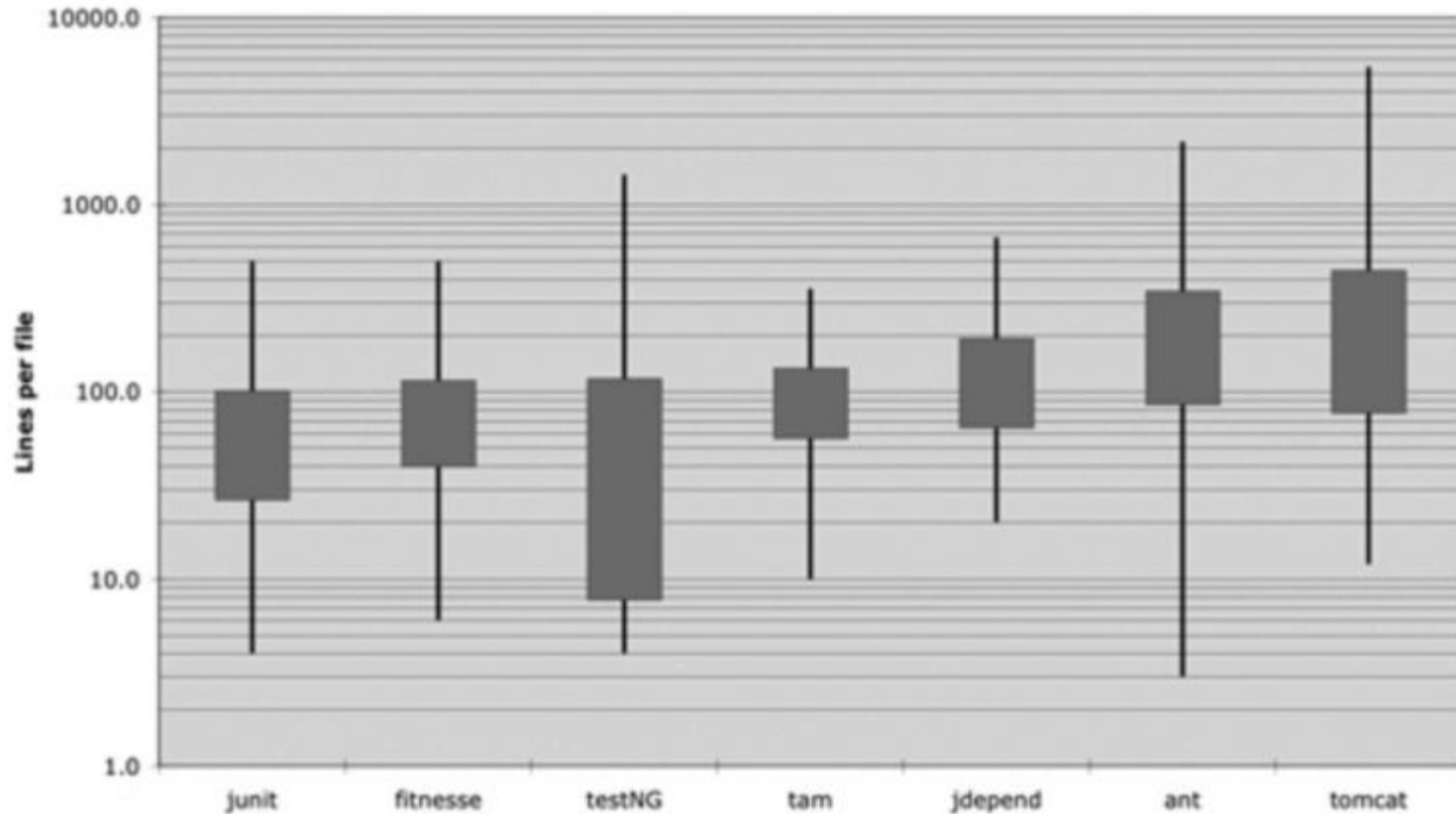


- Formatação vertical
- Separação de conceitos
- Funções dependentes
- Afinidade conceitual
- Formatação vertical
- Identação

Formatação vertical



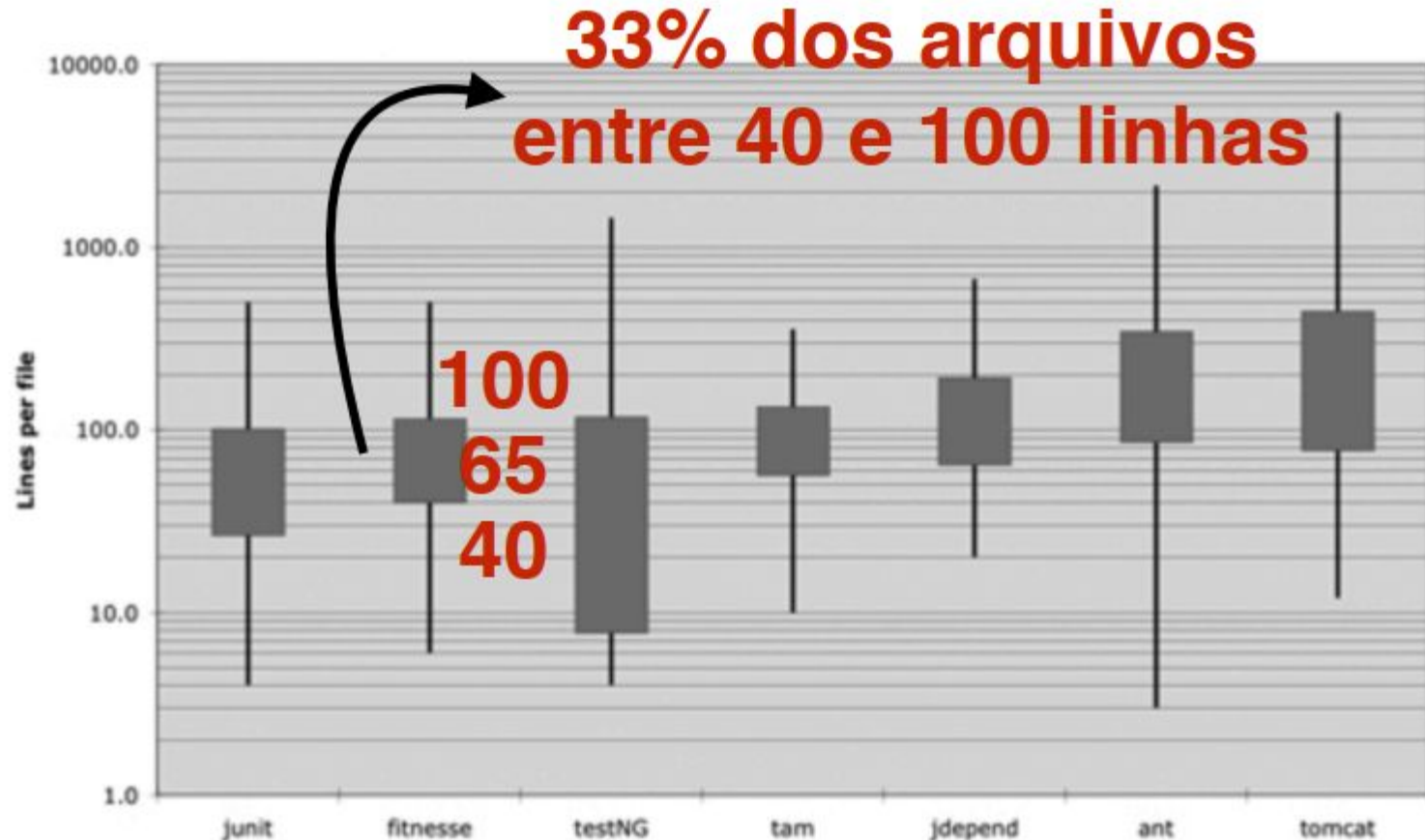
- Qual o tamanho um arquivo deve ter?



Formatação vertical



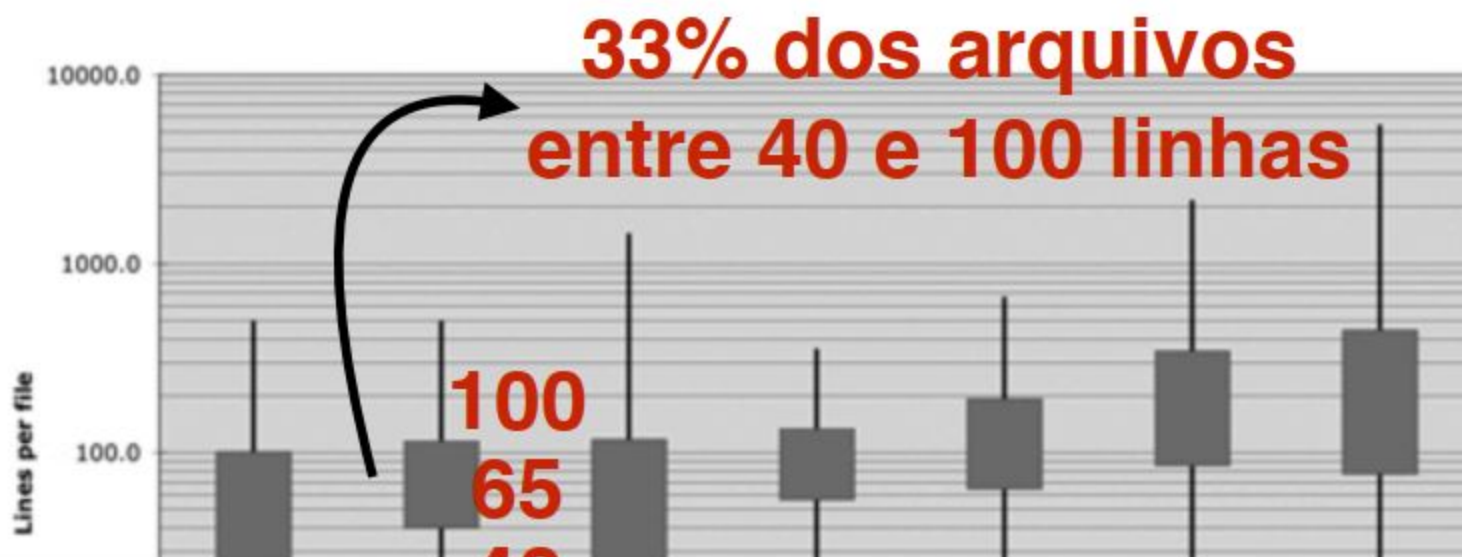
- Qual o tamanho um arquivo deve ter?



Formatação vertical



- Qual o tamanho um arquivo deve ter?



É possível construir sistemas reais com arquivos não muito longos

Formatação vertical

- Comparação: código x jornal
- Artigos são lidos verticalmente
- No topo: manchetes
- Primeiro parágrafo: sinopse
- Abaixo: detalhes



RS 1.50 Sexta-feira 8 de junho de 2018 Maringá, PR ANO XLIV - NÚMERO 13.527 Esta edição: 18 páginas, fechada às 23h30

O DIÁRIO
DO NORTE DO PARANÁ

clube do assinante
MUITO + VANTAGENS
tudo pensado para você
diario.com/clubedassinante

ESTATÍSTICAS

Mortes no trânsito têm queda de 43% com radares



● Desde a instalação dos 40 radares, foram registrados **9 óbitos** no trânsito de Maringá, ante 16 no mesmo período do ano passado ● A grande maioria das vítimas fatais são do **sexo masculino**

Letícia Conegero
leticia@diariom.com

Desde a instalação dos 40 radares que monitoram 80 faixas em Maringá, em fevereiro deste ano, foram registrados 9 óbitos no trânsito na cidade. O número corresponde ao mês de março, abril e maio, juntos. No mesmo período do ano passado, ocorreram 16 mortes. A redução é de 43,7%.

Tanto no ano passado, como neste ano, a maioria das vítimas fatais eram homens. Durante todo o ano de 2017, 44 pessoas morreram no perímetro urbano de Maringá - com exceção da Avenida Colombo, que é fiscalizada pela Polícia Rodoviária Federal (PRF). Destas, 38 eram do sexo masculino. Neste ano, de janeiro a maio, foram registrados 15 óbitos, dos quais 11 eram homens.

Segundo o diretor de Fiscalização da Secretaria de Mobilidade Urbana (Semob), Marcelo Filipe, o excesso de velocidade é uma das principais causas de acidentes de trânsito. "Na nota avaliada, o radar está cumprando o papel dele, que é reduzir os acidentes e óbitos na cidade. A gente percebe a presença de excesso de velocidade nos dois últimos acidentes com óbitos que ocorreram em maio, por exemplo", explica.

Segundo ele, essa diminuição também comprova a eficiência do estudo que foi feito acerca dos pontos com maior índice de acidentes. Mas, foi necessário esperar três meses para constatar a redução no número de óbitos. "Trata-se de um comparativo. Contudo, um veículo com cinco pessoas pode se envolver em um acidente e as cinco pessoas podem morrer. Ou seja, foi um único acidente com cinco óbitos. Por isso, esperamos um período maior para averiguar o real efeito dos radares nesta relação", enfatiza Filipe.

Ainda de acordo com o diretor de Fiscalização da Semob, a sociedade está condicionada a não obedecer as regras de trânsito quando há fiscalização. "Na verdade, não precisamos de radar. Devia ser uma questão de respeito e educação. Mas, quando não há fiscalização, percebemos um aumento de velocidade, pessoas sem volante, etc. E quando é feita a fiscalização, sem que seja por medo, elas deixam de cometer as irregularidades", declara.

A Semob vai analisar as estatísticas de trânsito para planejamento de ações. São verificadas informações sobre local, natureza dos acidentes e outras circunstâncias que levaram suas pessoas a perderem a vida.

O estudo contará com a parceria do Corpo de Bombeiros, das polícias militar, rodoviária estadual e federal, Instituto Médico Legal, Polícia Científica e Samu.

Formatação vertical



- Idealmente um arquivo de código deve ser igual a um artigo de jornal
- Nome: simples e explicativo
- Topo: funções de alto nível
- Mais abaixo: funções com mais detalhes

Separação de conceitos



- Cada grupo de linhas deve representar um conceito
- Regra extremamente simples

```
package fitnesse.wikitext.widgets;
import java.util.regex.*;
public class BoldWidget extends ParentWidget {
    public static final String REGEXP = "'.+?'";
    private static final Pattern pattern = Pattern.compile("'(.+?)'"
        Pattern.MULTILINE + Pattern.DOTALL);
    public BoldWidget(ParentWidget parent, String text) throws Exception {
        super(parent);
        Matcher match = pattern.matcher(text);
        match.find();
        addChildWidgets(match.group(1));
    }
    public String render() throws Exception {
        StringBuffer html = new StringBuffer("<b>");
        html.append(childHtml()).append("</b>");
        return html.toString();
    }
}
```

Separação de conceitos



```
package fitness.wikitext.widgets;

import java.util.regex.*;

public class BoldWidget extends ParentWidget {
    public static final String REGEXP = "'''.+?'";
    private static final Pattern pattern = Pattern.compile("'''(.+?)'
        Pattern.MULTILINE + Pattern.DOTALL
    );

    public BoldWidget(ParentWidget parent, String text) throws Except
        super(parent);
        Matcher match = pattern.matcher(text);
        match.find();
        addChildWidgets(match.group(1));
    }

    public String render() throws Exception {
        StringBuffer html = new StringBuffer("<b>");
        html.append(childHtml()).append("</b>");
        return html.toString();
    }
}
```

Funções dependentes



- Se uma função chama outra, elas devem estar próximas umas das outras (chamador acima da chamada)
- Fornece ao código um fluxo natural

```

public class WikiPageResponder implements SecureResponder {
    protected WikiPage page;
    protected PageData pageData;
    protected String pageTitle;
    protected Request request;
    protected PageCrawler crawler;

    public Response makeResponse(FitNesseContext context, Request request)
        throws Exception {
        String pageName = getPageNameOrDefault(request, "FrontPage");
        loadPage(pageName, context);
        if (page == null)
            return notFoundResponse(context, request);
        else
            return makePageResponse(context);
    }

    private String getPageNameOrDefault(Request request, String defaultPageName)
    {
        String pageName = request.getResource();
        if (StringUtil.isBlank(pageName))
            pageName = defaultPageName;

        return pageName;
    }

    protected void loadPage(String resource, FitNesseContext context)
        throws Exception {
        WikiPagePath path = PathParser.parse(resource);
        crawler = context.root.getPageCrawler();
        crawler.setDeadEndStrategy(new VirtualEnabledPageCrawler());
        page = crawler.getPage(context.root, path);
        if (page != null)
            pageData = page.getData();
    }

    private Response notFoundResponse(FitNesseContext context, Request request)
        throws Exception {
        return new NotFoundResponder().makeResponse(context, request);
    }
}

```





```
public class WikiPageResponder implements SecureResponder {
    protected WikiPage page;
    protected PageData pageData;
    protected String pageTitle;
    protected Request request;
    protected PageCrawler crawler;

    public Response makeResponse(FitNesseContext context, Request request)
        throws Exception {
        String pageName = 1 getPageNameOrDefault(request, "FrontPage");
        2 loadPage pageName, context);
        if (page == null)
            3 return new NotFoundResponse(context, request);
        else
            return makePageResponse(context);
    }

    private String getPageNameOrDefault(Request request, String defaultPageName)
    {
        String pageName = request.getResource();
        1 if (StringUtil.isBlank(pageName))
            pageName = defaultPageName;

        return pageName;
    }

    protected void loadPage(String resource, FitNesseContext context)
        throws Exception {
        2 WikiPagePath path = PathParser.parse(resource);
        crawler = context.root.getPageCrawler();
        crawler.setDeadEndStrategy(new VirtualEnabledPageCrawler());
        page = crawler.getPage(context.root, path);
        if (page != null)
            pageData = page.getData();
    }

    3 private Response notFoundResponse(FitNesseContext context, Request request)
        throws Exception {
        return new NotFoundResponder().makeResponse(context, request);
    }
}
```

Afinidade conceitual



- Dependência direta ou operações similares

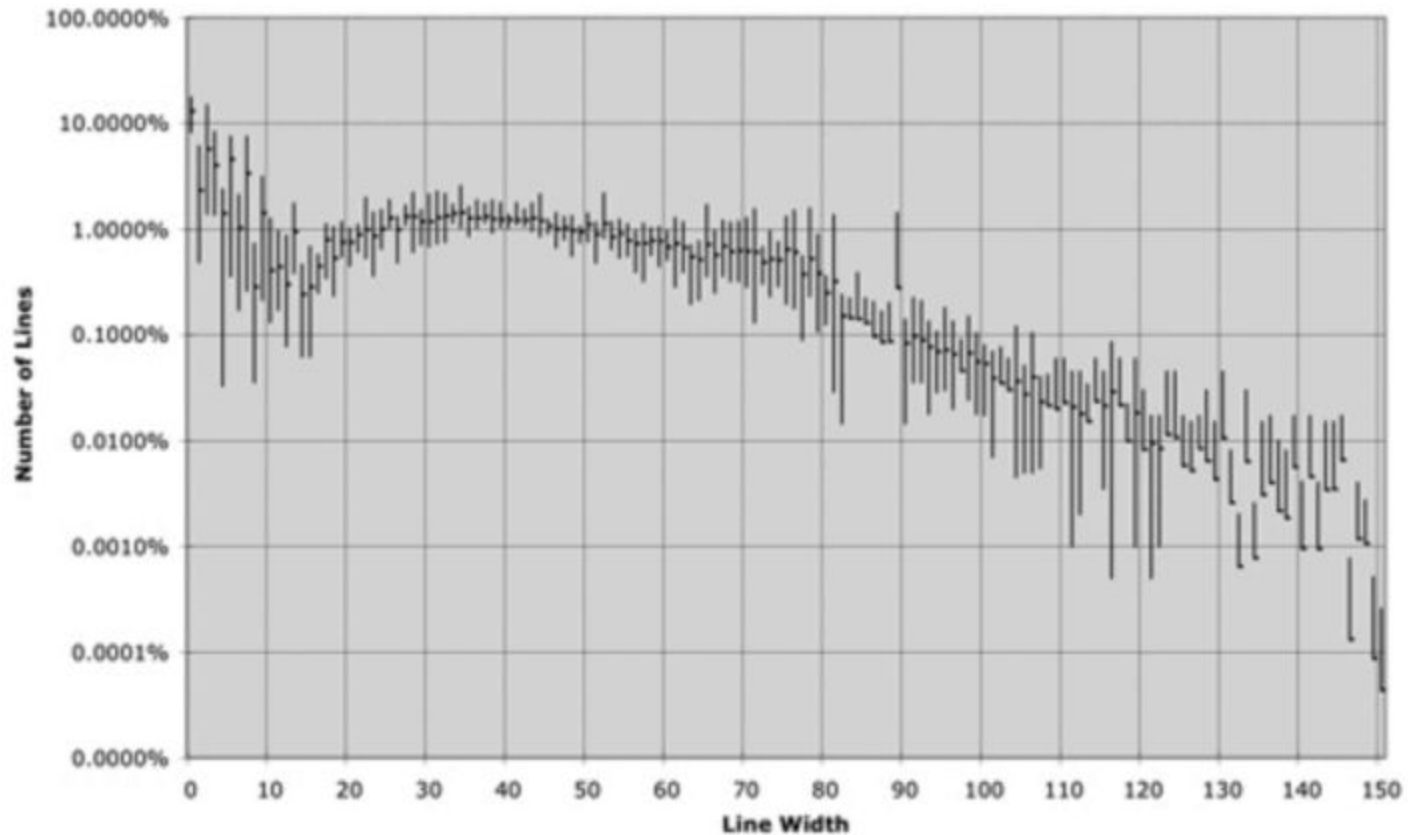
Variações
da mesma
tarefa
básica

```
public class Assert {  
    static public void assertTrue(String message, boolean condition) {  
        if (!condition)  
            fail(message);  
    }  
  
    static public void assertTrue(boolean condition) {  
        assertTrue(null, condition);  
    }  
  
    static public void assertFalse(String message, boolean condition) {  
        assertTrue(message, !condition);  
    }  
  
    static public void assertFalse(boolean condition) {  
        assertFalse(null, condition);  
    }  
    ...  
}
```


Formatação horizontal



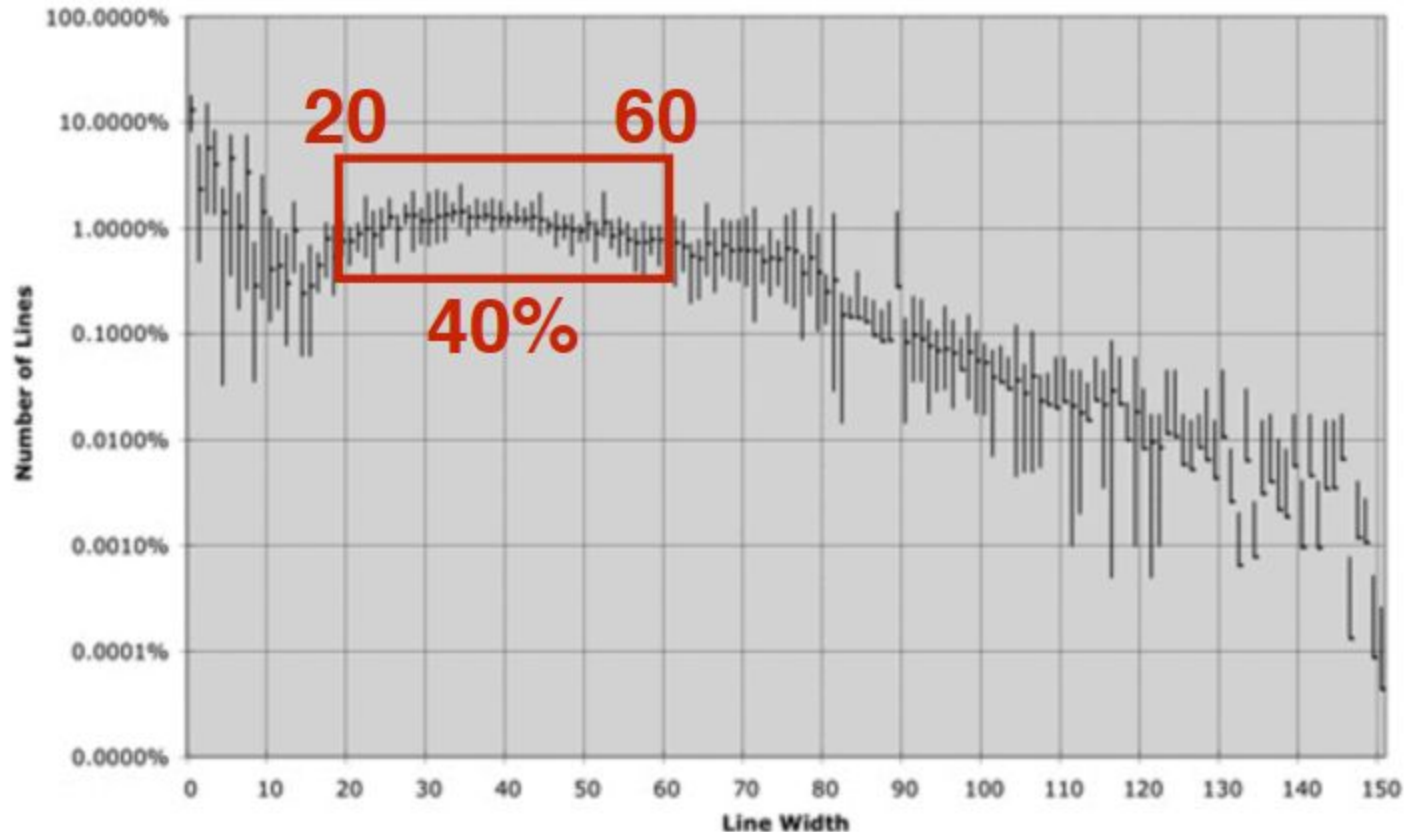
- O quão comprida uma linha deve ser?



Formatação horizontal



- O quão comprida uma linha deve ser?



Formatação horizontal



- O quão comprida uma linha deve ser?



Identação



- Código fonte é hierárquico
 - Classes, métodos, blocos de código, etc
 - Cada nível é um escopo, onde variáveis podem ser declaradas e utilizadas
- Identação: torna os níveis visíveis
 - Permite visualizar facilmente os escopos dos ifs, whiles, fors
- Sem identação: código tornam-se ilegíveis para humanos

```
public class FitNesseServer implements SocketServer { private FitNesseCont
context; public FitNesseServer(FitNesseContext context) { this.context =
context; } public void serve(Socket s) { serve(s, 10000); } public void
serve(Socket s, long requestTimeout) { try { FitNesseExpediter sender = ne
FitNesseExpediter(s, context);
sender.setRequestParsingTimeLimit(requestTimeout); sender.start(); }
catch(Exception e) { e.printStackTrace(); } } }
```



```
public class FitNesseServer implements SocketServer { private FitNesseCont
context; public FitNesseServer(FitNesseContext context) { this.context =
context; } public void serve(Socket s) { serve(s, 10000); } public void
serve(Socket s, long requestTimeout) { try { FitNesseExpediter sender = ne
FitNesseExpediter(s, context);
sender.setRequestParsingTimeLimit(requestTimeout); sender.start(); }
catch(Exception e) { e.printStackTrace(); } } }
```



```
public class FitNesseServer implements SocketServer {
    private FitNesseContext context;

    public FitNesseServer(FitNesseContext context) {
        this.context = context;
    }

    public void serve(Socket s) {
        serve(s, 10000);
    }

    public void serve(Socket s, long requestTimeout) {
        try {
            FitNesseExpediter sender = new FitNesseExpediter(s, context);
            sender.setRequestParsingTimeLimit(requestTimeout);
            sender.start();
        }
        catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```



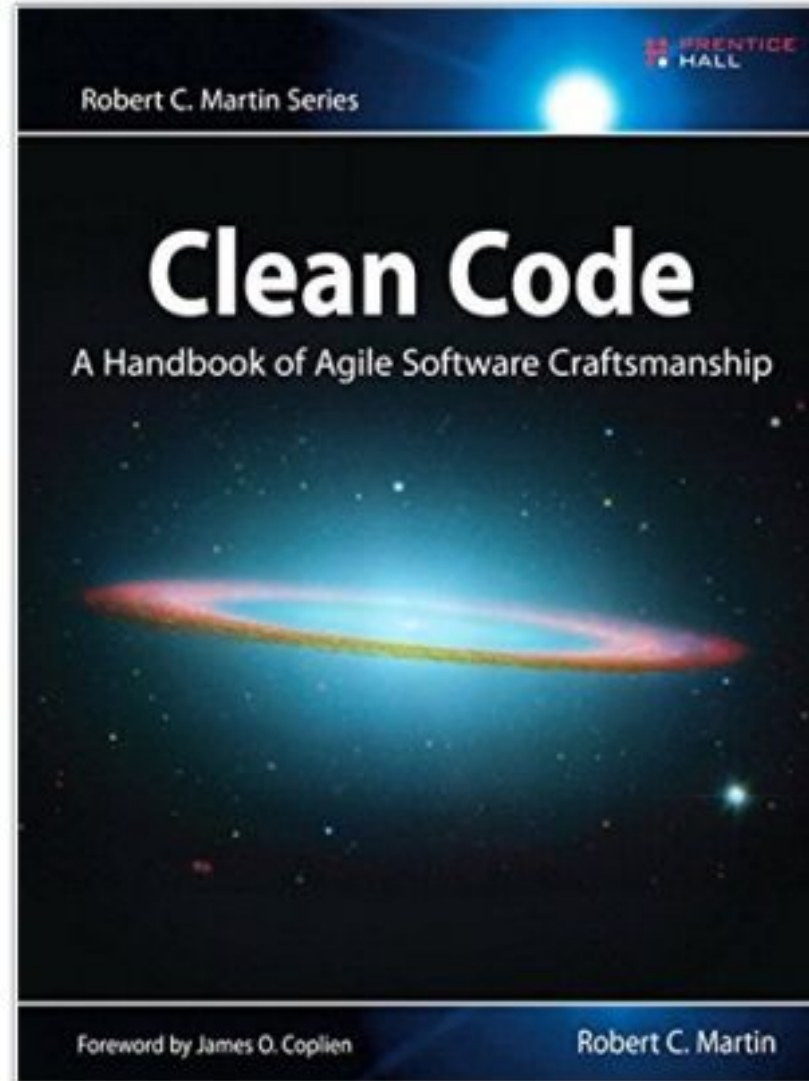
Manutenção de Software

Código Limpo

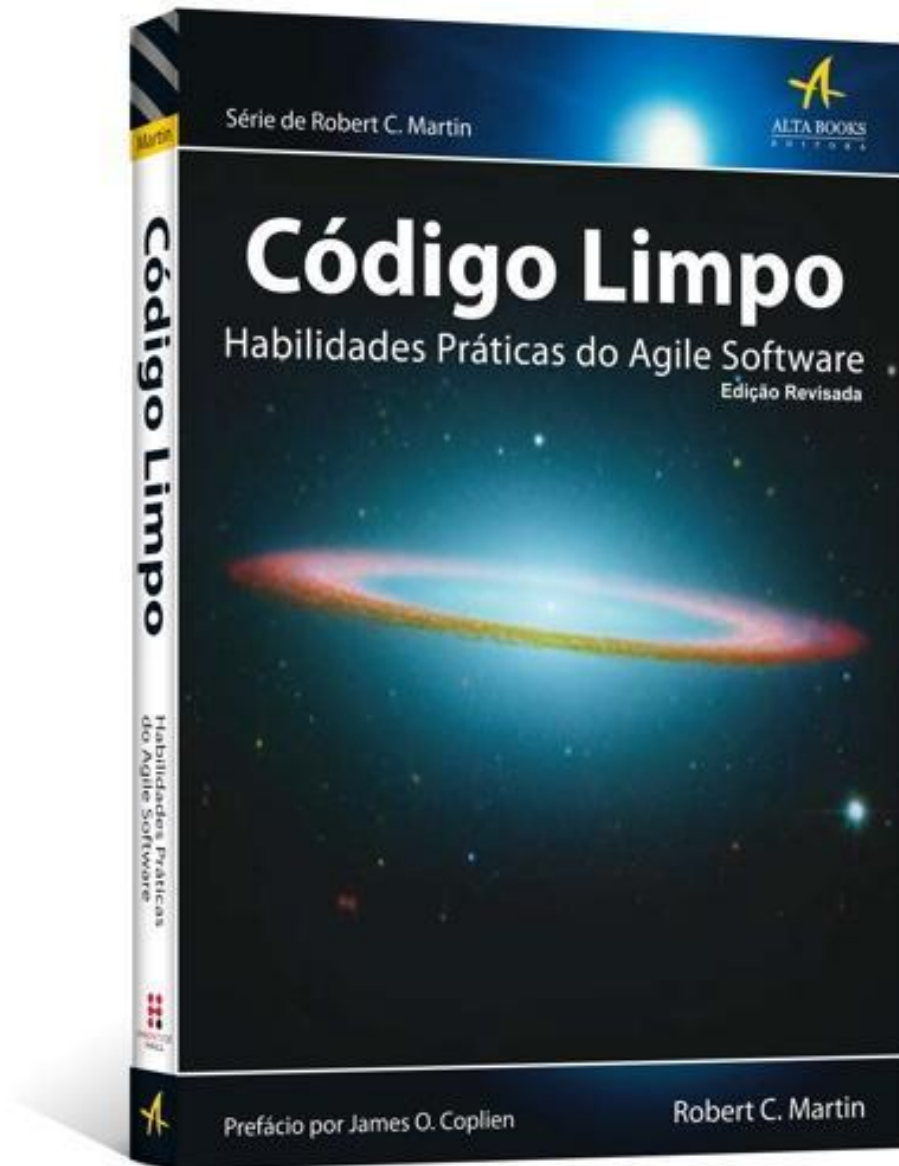


- **Legibilidade**
- **Comentários**
- **Formatação**

Código Limpo



Código Limpo



Código Limpo



“Você está lendo este livro por duas razões. Primeiro, você é um programador. Segundo, você quer ser um programador melhor. Ótimo. Precisamos de programadores melhores.”

O que é Código Limpo?



“Código limpo é simples e direto. Código limpo se lê como uma prosa bem escrita. Código limpo nunca obscurece a intenção do projetista; ao contrário, é repleto de abstrações e de fluxos de controle claros e diretos.

Grady Booch – criador da UML

O que é Código Limpo?



“Gosto que meu código seja **elegante e eficiente**. A lógica deve ser direta, para dificultar que bugs se escondam; as dependências devem ser mínimas, para facilitar a manutenção; o tratamento de erros deve ser completo, de acordo com uma estratégia bem definida; e o desempenho deve estar próximo do ideal, para não tentar as pessoas a bagunçar o código com otimizações sem critério. Código limpo faz uma coisa bem feita.”

Bjarne Stroustrup – criador do C++

O que é Código Limpo?



“Código limpo pode ser lido e aprimorado por um desenvolvedor que não seja seu autor original. Ele possui testes unitários e de aceitação. Tem nomes significativos. Oferece uma forma, em vez de várias, de fazer a mesma coisa. Possui dependências mínimas, explicitamente definidas, e fornece uma API clara e enxuta.”

Dave Thomas – criador do Eclipse

Código Limpo



- Legibilidade
- Comentários
- Formatação
- Funções
- Código externo
- Testes de unidade
- Classes

Código Limpo



- **Legibilidade**
- **Comentários**
- **Formatação**
- Funções
- Código externo
- Testes de unidade
- Classes

Código Limpo - Legibilidade



- Nomes são tudo em um software
- Nomeamos variáveis, métodos, argumentos, classes, pacotes, arquivos, diretórios...

Dicas de Legibilidade



- Usar nomes que revelam a intenção
- Fazer distinção entre nomes
- Usar nomes pronunciáveis
- Usar nomes “buscáveis”

O que é Código Limpo?



```
for (int j=0; j<34; j++) {  
    s += (t[j]*4)/5;  
}
```

O que é Código Limpo?



```
int realDaysPerIdealDay = 4;
const int WORK_DAYS_PER_WEEK = 5;
int sum = 0;
for (int j=0; j < NUMBER_OF_TASKS; j++) {
    int realTaskDays = taskEstimate[j] * realDaysPerIdealDay;
    int realTaskWeeks = (realdays / WORK_DAYS_PER_WEEK);
    sum += realTaskWeeks;
}
```

Dicas de Legibilidade



- Usar nomes que revelam a intenção
 - `int d; // tempo em dias`
 - `int s; // semanas`
 - `int m; // meses`
 - `int a; // anos`

Dicas de Legibilidade



- Usar nomes que revelam a intenção
- **int tempoEmDias;**
- **int diasDesdeACriacao;**
- **int idadeDoArquivoEmDias;**

Dicas de Legibilidade



- Usar nomes que revelam a intenção

```
public List<int[]> getThem() {  
    List<int[]> list1 = new ArrayList<int[]>();  
    for (int[] x : theList)  
        if (x[0] == 4)  
            list1.add(x);  
    return list1;  
}
```

Dicas de Legibilidade



- Usar nomes que revelam a intenção

```
public List<int[]> getFlaggedCells() {  
    List<int[]> flaggedCells = new ArrayList<int[]>();  
    for (int[] cell : gameBoard)  
        if (cell[STATUS_VALUE] == FLAGGED)  
            flaggedCells.add(cell);  
    return flaggedCells;  
}
```


Dicas de Legibilidade



- Usar nomes que revelam a intenção

```
public List<Cell> getFlaggedCells() {  
    List<Cell> flaggedCells = new ArrayList<Cell>();  
    for (Cell cell : gameBoard)  
        if (cell.isFlagged())  
            flaggedCells.add(cell);  
    return flaggedCells;  
}
```

Dicas de Legibilidade



- Fazer distinção entre nomes

```
public static void copyChars(char a1[], char a2[]) {  
    for (int i = 0; i < a1.length; i++) {  
        a2[i] = a1[i];  
    }  
}
```

Dicas de Legibilidade



- Fazer distinção entre nomes
- Copia o que para onde?
- Definir fonte e destino (a1/a2)

```
public static void copyChars(char a1[], char a2[]) {  
    for (int i = 0; i < a1.length; i++) {  
        a2[i] = a1[i];  
    }  
}
```

Dicas de Legibilidade



- Usar nomes pronunciáveis

```
class DtaRcrd102 {  
    private Date genymdhms;  
    private Date modymdhms;  
    private final String pszqint = "102";  
    /* ... */  
};
```

Dicas de Legibilidade



- Usar nomes pronunciáveis

```
class Customer {  
    private Date generationTimestamp;  
    private Date modificationTimestamp;  
    private final String recordId = "102";  
    /* ... */  
};
```

Dicas de Legibilidade



- Usar nomes “buscáveis”

```
for (int j=0; j<34; j++) {  
    s += (t[j]*4)/5;  
}
```

Dicas de Legibilidade



- Usar nomes “buscáveis”

```
int realDaysPerIdealDay = 4;
const int WORK_DAYS_PER_WEEK = 5;
int sum = 0;
for (int j=0; j < NUMBER_OF_TASKS; j++) {
    int realTaskDays = taskEstimate[j] * realDaysPerIdealDay;
    int realTaskWeeks = (realdays / WORK_DAYS_PER_WEEK);
    sum += realTaskWeeks;
}
```

Código Limpo – Comentários



- Não existe nada mais útil do que um bom comentário
- Não existe nada pior do que um comentário ruim, redundante, desnecessário...
- O propósito dos comentários é compensar a falha de nos expressarmos em código

Código Limpo – Comentários



- Qual sua opinião sobre o código abaixo?

```
// Check to see if the employee is eligible for full benefits  
if ((employee.flags & HOURLY_FLAG) &&  
    (employee.age > 65))
```

- Tem como melhorar?

Código Limpo – Comentários



- Qual sua opinião sobre o código abaixo?

```
// Check to see if the employee is eligible for full benefits  
if ((employee.flags & HOURLY_FLAG) &&  
    (employee.age > 65))
```

- Tem como melhorar?

```
if (employee.isEligibleForFullBenefits())
```

Comentários relevantes



- Qual sua opinião sobre o código abaixo?

```
// Returns an instance of the Responder being tested.  
protected abstract Responder responderInstance();
```

- Tem como melhorar?

Comentários relevantes



- Qual sua opinião sobre o código abaixo?

```
// Returns an instance of the Responder being tested.  
protected abstract Responder responderInstance();
```

- Tem como melhorar?

responderBeingTested()

Comentários relevantes



- Qual sua opinião sobre o código abaixo?

```
// format matched kk:mm:ss EEE, MMM dd, yyyy  
Pattern timeMatcher = Pattern.compile(  
    "\\d*:\\d*:\\d* \\w*, \\w* \\d*, \\d*");
```

- Tem como melhorar?



Comentários relevantes

- Qual sua opinião sobre o código abaixo?

```
// format matched kk:mm:ss EEE, MMM dd, yyyy  
Pattern timeMatcher = Pattern.compile(  
    "\\d*:\\d*:\\d* \\w*, \\w* \\d*, \\d*");
```

- Tem como melhorar?
- **Mover código para uma classe que trata de datas e tempo – elimina o comentário**

Comentários redundantes



```
/** The name. */  
private String name;  
  
/** The version. */  
private String version;  
  
/** The licenceName. */  
private String licenceName;  
  
/** The version. */  
private String info;
```

Comentários redundantes



```
/** The day of the month. */  
private int dayOfMonth;
```


Comentários redundantes



```
/**  
 * Default constructor.  
 */  
protected AnnualDateRule() {  
}
```

Comentários redundantes



```
/**
 * Returns the day of the month.
 *
 * @return the day of the month.
 */
public int getDayOfMonth() {
    return dayOfMonth;
}
```

Comentários redundantes



```
public abstract class ContainerBase
    implements Container, Lifecycle, Pipeline,
    MBeanRegistration, Serializable {

    /**
     * The processor delay for this component.
     */
    protected int backgroundProcessorDelay = -1;

    /**
     * The lifecycle event support for this component.
     */
    protected LifecycleSupport lifecycle =
        new LifecycleSupport(this);

    /**
     * The container event listeners for this Container.
     */
    protected ArrayList listeners = new ArrayList();

    /**
     * The Loader implementation with which this Container is
     * associated.
     */
    protected Loader loader = null;
```

Comentários desatualizados



```
MockRequest request;
private final String HTTP_DATE_REGEX =
    "[SMTWF][a-z]{2}\\,\\s[0-9]{2}\\s[JFMASOND][a-z]{2}\\s"+
    "[0-9]{4}\\s[0-9]{2}\\:[0-9]{2}\\:[0-9]{2}\\sGMT";
private Response response;
private FitNesseContext context;
private FileResponder responder;
private Locale saveLocale;
// Example: "Tue, 02 Apr 2003 22:18:49 GMT"
```

Código Comentado



```
InputStreamResponse response = new InputStreamResponse();
response.setBody(formatter.getResultStream(), formatter.getByteCount());
// InputStream resultsStream = formatter.getResultStream();
// StreamReader reader = new StreamReader(resultsStream);
// response.setContent(reader.read(formatter.getByteCount()));
```

Código Comentado



```
while (!stack.isEmpty()) {  
    State current = stack.pop();  
  
    // int theDepth = current.getDepth();  
    //  
    // if(theDepth > maxDepthSearched)  
    //     maxDepthSearched = theDepth;  
  
    if (current.isGoal(maze)){  
        // TODO update the maze if a solution found  
        //while(current != null){  
        while(current.getParent() != null){  
            if(!current.isGoal(maze))  
                maze.setOneSquare(current.getSquare(), '.');  
            current = current.getParent();  
        }  
        return true;  
    }  
}
```

Commented-out code:
Código “removido”
através de comentário

Desvantagens de código comentado



- Código comentado é uma distração e pode confundir o programador
- Código comentado sempre levanta mais questões do que respostas
- Código comentado torna-se defasado ao longo do tempo uma vez que o sistema continua a evoluir enquanto o trecho de código comentado permanece estático

Desvantagens de código comentado



- Código comentado é uma forma errada de controle de versão, uma vez que ferramentas como Git e SVN são indiscutivelmente a solução apropriada
- O simples fato do programador buscar entender a razão por trás do código comentado pode tomar muito tempo
- Programadores esquecerão rapidamente o quão relevante é o código comentado